

Modern adatbázis rendszerek

MSc

2025

Készítette:
Szendrei Gábor
V9ZK10

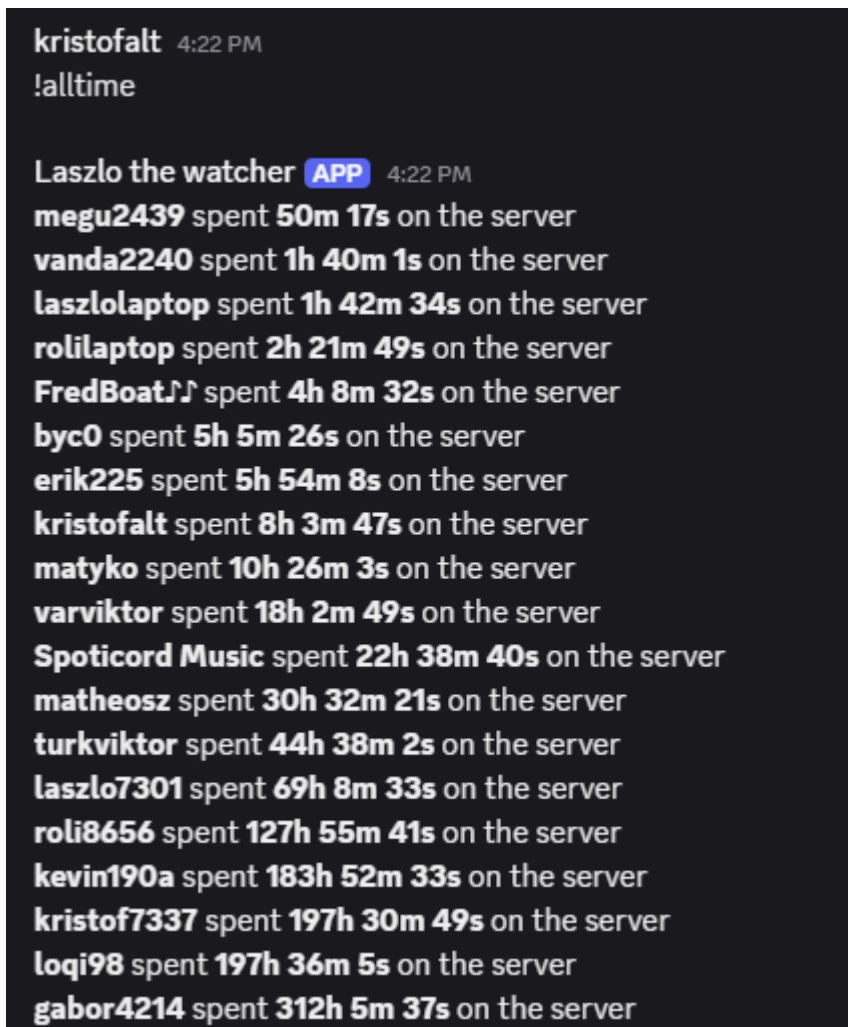
DiscordActivityBot Dokumentáció

A **DiscordActivityBot** egy olyan bot, amely a Discord szerverek felhasználói aktivitását követi nyomon, és részletes statisztikákat biztosít. Ez a dokumentáció részletesen bemutatja a bot működését, különös tekintettel a MongoDB használatára.

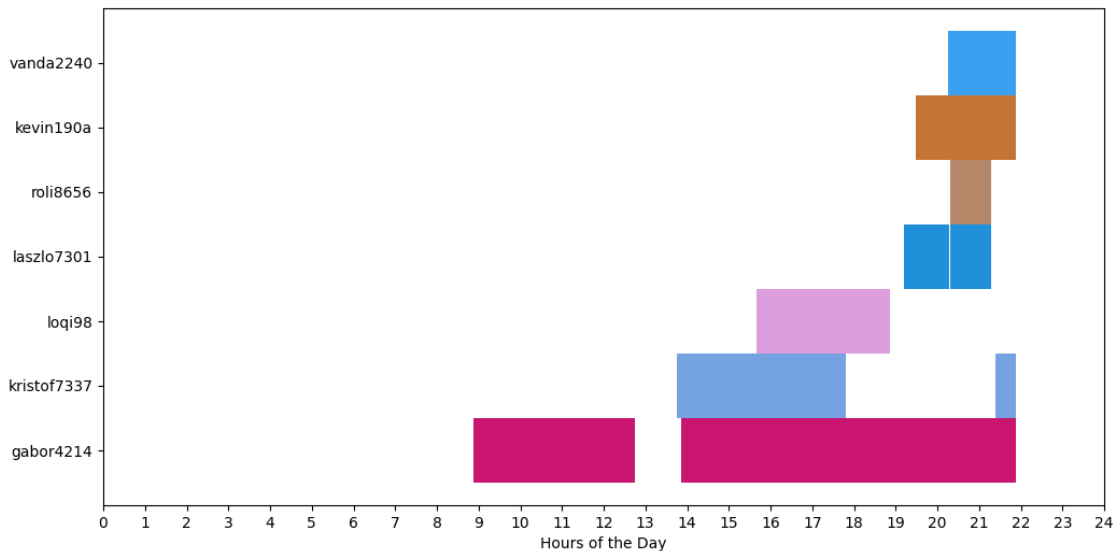
Áttekintés

A bot célja, hogy a szerver adminisztrátorai számára betekintést nyújtson a felhasználók aktivitásába. A bot a következő funkciókat biztosítja:

- **Felhasználói aktivitás követése:** Nyomon követi, hogy a felhasználók mennyi időt töltenek a szerveren.



- **Uptime jelentések:** Megmutatja, hogy az éppen online lévő felhasználók mennyi ideje aktívak.
- **Gantt-diagramok generálása:** Vizualizálja a napi aktivitást Gantt-diagram formájában.



A bot MongoDB-t használ az adatok tárolására, amely lehetővé teszi a nagy mennyiségű adat hatékony kezelését és lekérdezését.

MongoDB Használata

A bot működésének központi eleme a MongoDB, amely egy NoSQL típusú adatbázis. A MongoDB dokumentum-alapú adatmodellt használ, amely JSON-szerű dokumentumokban tárolja az adatokat. Ez a rugalmasság ideális a DiscordActivityBot számára, mivel a felhasználói aktivitás adatai dinamikusak és változatosak lehetnek.

Adatbázis Struktúra

A bot a MongoDB-ben több gyűjteményt (collection) használ az adatok tárolására. Az alábbiakban bemutatjuk a főbb gyűjteményeket és azok szerkezetét:

1. users Gyűjtemény

Ez a gyűjtemény a szerver felhasználóinak adatait tárolja.

```
{
  "_id": "123456789012345678", // Felhasználó Discord ID-ja
  "username": "JohnDoe",       // Felhasználó neve
  "total_time": 3600,           // Összes idő másodpercben, amit a szerveren töltött
  "sessions": [                // Minden egyes aktivitási szakasz
    {
      "start_time": "2025-04-01T10:00:00Z", // Aktivitás kezdete
      "end_time": "2025-04-01T12:00:00Z"   // Aktivitás vége
    }
  ]
}
```

2. activity_logs Gyűjtemény

```
{
  "_id": "event12345",
  "user_id": "123456789012345678", // Felhasználó Discord ID-ja
  "event_type": "join",              // Esemény típusa: "join" vagy "leave"
  "timestamp": "2025-04-01T10:00:00Z" // Esemény időbélyege
}
```

Ez a gyűjtemény a szerver aktivitási eseményeit tárolja.

3. charts Gyűjtemény

Ez a gyűjtemény a generált Gantt-diagramok metaadatait tárolja.

```
{
  "_id": "chart20250401",
  "date": "2025-04-01",           // A diagramhoz tartozó dátum
  "generated_at": "2025-04-01T23:59:59Z", // Generálás időpontja
  "chart_url": "https://example.com/chart20250401.png" // Diagram URL-je
}
```

MongoDB Műveletek

A bot a MongoDB-vel különböző CRUD (Create, Read, Update, Delete) műveleteket végez. Az alábbiakban bemutatom a legfontosabb műveleteket.

1.Inicializáció:

```
class MongoDB:
    _initiated = None
    def __init__(self):
        self.client = MongoClient(os.getenv('MONGO_CON'))
        self.database = self.client.discord
    def get_collection(self, collection):
        return self.database[collection]

def MongoInit():
    if MongoDB._initiated is None:
        MongoDB._initiated = MongoDB()
    return MongoDB._initiated
```

2. Aktivitás Rögzítése

Amikor egy felhasználó belép vagy kilép a szerverről, az eseményt rögzíti az activity_logs gyűjteményben.

```
def __del__(self):
    session_data : dict = {
        "_id": self.id,
        "server_id": self.server_id,
        "channel": self.channel,
        "user": self.user,
        "start": self.session_start,
        "end": datetime.now() + timedelta(hours=1)
    }

    insert_result = self.mongo.get_collection("sessions").insert_one(session_data)

    if insert_result.inserted_id:
        print(f"{self.user} left {self.channel}")

    self._update_mic_state_end()
```

3. Összesített Idő Frissítése

A bot kiszámítja a felhasználó által a szerveren töltött időt, és frissíti a `total_time` mezőt.

```
const updateTotalTime = async (userId, session) => {
  const duration = (new Date(session.end_time) - new Date(session.start_time)) / 1000;
  await db.collection('users').updateOne(
    { _id: userId },
    {
      $inc: { total_time: duration },
      $push: { sessions: session }
    }
  );
};
```

4. Gantt-Diagram Generálása

A bot a napi aktivitási adatokat lekéri, és egy külső könyvtár segítségével Gantt-diagramot generál.

```
def makeReport(self, server_id, day, sessions):
    next_day = day + timedelta(days=1)
    result = self.mongo.get_collection("sessions").find({
        "server_id": server_id,
        "$or": [
            {"start": {"$lt": next_day, "$gte": day}},
            {"end": {"$gt": day, "$lt": next_day}},
            {"start": {"$lte": day}, "end": {"$gte": next_day}}
        ]
    })
    result = list(result)
```

Bot Parancsok

A bot az alábbi parancsokat támogatja:

- **!alltime:** Lekéri az összes felhasználó által a szerveren töltött időt.
- **!uptime:** Megjeleníti az éppen online lévő felhasználók aktivitási idejét.
- **!gant:** Generál egy Gantt-diagramot az aktuális napi aktivitásról.

Telepítés és Konfiguráció

1. MongoDB Beállítása

- Hozz létre egy MongoDB adatbázist (pl. discord_activity néven).
- Állítsd be a MongoDB URI-t az .env fájlban:

```
MONGO_URI=mongodb+srv://<felhasználónév>:<jelszó>@cluster0.mongodb.net/discord_activity?retryWrites=true&w=majority
```

2. Bot Futtatása

1. Telepítsd a szükséges csomagokat:

```
npm install
```

2. Indítsd el a botot:

```
node index.js
```

Összegzés

A DiscordActivityBot hatékony eszköz a szerver aktivitásának nyomon követésére. A MongoDB rugalmassága és teljesítménye lehetővé teszi a nagy mennyiségű adat kezelését, miközben a bot egyszerű és intuitív parancsokat biztosít a felhasználók számára.

Miskolc,
2025